

3. TECHNICAL DOCUMENTATION

1. Introduction

Retrieval-Augmented Generation (RAG) is a hybrid approach that combines information retrieval with natural language generation.

Traditional LLM-based systems generate responses based solely on learned patterns, which can lead to hallucinations. RAG addresses this limitation by retrieving relevant information from a trusted knowledge base and using it as context for generating responses.

This ensures:

- Higher factual accuracy
- Reduced hallucination
- Domain-specific responses

2. System Architecture

The system follows a modular architecture consisting of:

- **Data Ingestion Layer:** Processes PDF and generates embeddings
- **Storage Layer:** Stores vector embeddings in ChromaDB
- **Retrieval Layer:** Performs semantic search to find relevant content
- **Processing Layer:** Uses LLM to generate responses
- **Control Layer:** LangGraph orchestrates workflow and decision-making
- **Human Layer (HITL):** Handles complex queries

This layered approach improves maintainability, scalability, and reliability.

3. Design Decisions

Chunk Size = 500

- Balances context richness and efficiency
- Too large → slower retrieval
- Too small → loss of meaning

Overlap = 100

- Maintains continuity between chunks
- Prevents loss of important context

Top-K = 3

- Ensures only relevant chunks are retrieved
- Avoids overloading LLM with unnecessary data

MiniLM Embeddings

- Lightweight and fast
- Suitable for CPU-based execution

Groq LLM

- High-speed inference
- Low latency responses

4. Workflow Explanation

The workflow is implemented using LangGraph as a directed graph of nodes.

Each node represents a processing step:

- Input Node → retrieves context
- Router Node → decision-making
- Answer Node → generates response
- HITL Node → human intervention
- Output Node → formats response

This structured pipeline ensures consistent and traceable execution.

5. Conditional Logic

The system uses rule-based and LLM-based validation to determine the appropriate response path.

Conditions include:

1. Absence of relevant context
2. Detection of sensitive keywords
3. LLM confidence evaluation

If any condition fails, the system routes the query to a human agent.

6. HITL Implementation

The Human-in-the-Loop (HITL) mechanism ensures reliability in critical scenarios.

When triggered:

- System pauses automated processing
- A human agent provides response via CLI
- Response is returned to the user

This improves trust and handles edge cases effectively.

7. Challenges & Trade-offs

Model Deprecation

- Required updating LLM model
- Shows dependency on external APIs

Accuracy vs Speed

- More context = better answers but slower

Chunk Size Trade-off

- Larger chunks = more meaning
- Smaller chunks = faster retrieval

Cost vs Performance

- Local embeddings reduce cost
- LLM usage still requires API

8. Testing Strategy

The system was tested across multiple scenarios:

- **Normal Queries** - Valid factual questions
- **Troubleshooting Queries** - Problem-solving scenarios
- **Unknown Queries** - Out-of-scope inputs
- **Escalation Scenarios** - Sensitive or complex queries

This ensures robustness and reliability.

9. Future Enhancements

- Multi-document ingestion
- Web-based UI using Streamlit
- Chat memory for conversation context
- Deployment using cloud services
- Integration with ticketing systems